Maintenance triage that puts safety *first*

What I built to triage maintenance requests safely, the two failure loops that broke, and what I learned.

Real emergencies don't label themselves

01 **Squeaky hinge**
Marked urgent by the tenant, but needs a routine handyman.

02 **Hidden gas leak**
Polite and understated note, but needs an immediate contractor.

Tenants describe problems in different ways. A squeaky door gets marked urgent, while a gas leak arrives in an apology.

From messy intake to reviewed dispatch

BEFORE MANAGER REVIEW

Incoming request → **Clarify details** → **Assess urgency**

AFTER MANAGER REVIEW

Manager decides → **Vendor notified** → **Tenant updated**

Advisory only: the workflow never dispatches a vendor without human approval.

I built a workflow that gathers missing details, separates genuine hazards from routine repairs, and presents ready-to-approve dispatches to a manager.

AI advises. The manager authorizes.

AI assists

Extract symptoms

Identify trade

Draft dispatch

Manager decides

Verify diagnosis

Select contractor

Approve each dispatch

AI handles the heavy reading, symptom extraction, and draft wording. The property manager verifies the repair and authorizes every contractor dispatch.

Safety rules can escalate, never down-rank

01 **Hazard detection**
Gas odors, spreading water, or sparking electrical signals.

02 **Forced escalation**
Safety rules immediately override priority to emergency.

03 **No down-ranking**
Software is never permitted to dismiss a danger signal.

If a message mentions a gas smell or flooding, deterministic rules force an immediate emergency alert. The automation cannot downgrade a safety risk.

The system emailed itself

THE GAP

Vendor dispatch → **Shared inbox** → ~~**Duplicate ticket**~~

Outbound dispatch re-entered intake

THE FIX

Vendor dispatch → **System tag** → **Filtered out**

Intake triggers ignore all outgoing system emails.

Outbound contractor emails landed in the same inbox and triggered new maintenance tickets. Adding system tags to outgoing email subjects stopped the loop permanently.

A status question became a new repair

THE BUG

"Any update?"	→ Triage engine	→ ~~New ticket opened~~

THE FIX

"Any update?"	→ Check open repairs	→ Factual status reply

Status queries are answered directly without disturbing the manager.

When a tenant asked when their repair would finish, the system opened another ticket. An upfront status check now returns existing progress instead.

Uncertainty needs a person

WHAT COMES IN	WHAT HAPPENS
Disability accommodation	Direct human review
Legal or lease question	Route to staff
Unclear symptom	Ask tenant clarification
AI timeout or failure	Flag for attention

Disability accommodations, legal questions, and vague complaints bypass automated drafting. If the model fails or times out, the request is flagged for staff.

Edge cases are the real build

The rambling message

↓

The hidden hazard

↓

The second click

↓

The email feedback loop

The hardest part of maintenance triage is not extracting symptoms. It is protecting the workflow against feedback loops, duplicate button clicks, and ambiguous wording.

I checked the awkward cases

I CHECKED	RECORDED BEHAVIOR
Understated gas odor	Escalated to emergency
Squeaky door marked urgent	Kept as routine
Approve tapped twice	No second email sent
Follow-up status query	Answered without a ticket

I checked understated safety hazards, routine issues marked urgent, duplicate approvals, and follow-up status queries to ensure predictable behavior.

What I'd improve next

01 Structured photo collection

Ask tenants for photos before dispatching expensive trades.

02 Contractor ETA tracking

Track vendor confirmations and arrival windows automatically.

03 Property system sync

Sync work order status and invoices directly with accounting.

Next, I would add guided photo uploads for ambiguous leaks and two-way contractor arrival tracking, reducing unnecessary callouts even further.